

Characterizing Single-GPU Decoder-Only Transformer Inference: Precision, Compilation, Batching, and Sequence Length on an RTX 5070

Ben Karlsberg

Abstract

We present a controlled single-GPU study of decoder-only transformer inference using a small open causal language model. On an NVIDIA GeForce RTX 5070, we measure synchronized end-to-end `generate()` latency and tokens-per-second across a 32-cell matrix spanning data type (FP16, BF16), batch size (1, 4), prompt length (64, 256), generation length (32, 128), and execution mode (eager vs. `torch.compile`). All cells completed without out-of-memory failures. BF16 reduced end-to-end latency relative to FP16 in every paired comparison (median BF16/FP16 latency ratio 0.90). Increasing batch size from 1 to 4 raised throughput by roughly $4\times$ while keeping wall-clock latency in a similar range. A follow-up batch-size sweep (1 to 256) locates the latency-throughput knee: latency stays near 2.3s up to batch 32 while throughput rises about $31\times$, after which latency grows roughly linearly with batch size and throughput levels off near 2,400 tok/s. Generation length dominated end-to-end time, while tokens-per-second stayed comparatively flat, consistent with decode-bound behavior under this harness. `torch.compile` yielded a median eager/compile latency ratio of 1.02 (range 0.92–1.22), indicating little systematic gain for this model and measurement window. We release the harness, configurations, and aggregated results at <https://github.com/benkarlsberg/GPU-Optimized-Transformer-Inference>.

1 Introduction

Autoregressive transformer inference [1] is commonly deployed with mixed precision, compiler stacks, and request batching. Practitioners must choose among these options without assuming that every optimization helps every workload. This paper reports a small, reproducible characterization on one consumer GPU using the open-source `goti` harness accompanying this work. Our goal is empirical clarity on a fixed model and hardware point—not a claim of state-of-the-art serving performance.

Serving systems batch requests on the premise that small-batch decoding leaves the GPU underused [20, 12, 14]. How far batching stays free before latency rises depends on the model and the device, and is rarely reported for small models on consumer GPUs through the standard Hugging Face `generate()` path. Our contributions are (i) a 32-cell factorial measurement of precision, batch size, prompt length, generation length, and `torch.compile` for a 135M-parameter model on an RTX 5070; (ii) a batch-size sweep from 1 to 256 that locates the latency-throughput knee, with an explicit account of an excluded noisy pass; and (iii) an open harness that records the configuration and environment of every run.

2 Related work

Serving systems and batching. Orca [20] introduced iteration-level scheduling so that new requests can join a running batch, and vLLM [12] stores the KV cache in fixed-size pages so that more sequences fit in memory at once. Sarathi-Serve [3] studies the throughput–latency tradeoff created when prefill and decode

share a batch. Pope et al. [14] model inference cost as a function of batch size, memory bandwidth, and partitioning on TPUs, and DeepSpeed-Inference [5] and FlexGen [17] target models too large for a single device or its memory. These systems motivate batching; our sweep measures where its benefit ends for a small model, using static batches rather than a serving engine.

Kernels and attention. FlashAttention [8, 7] reduces attention memory traffic by tiling the computation, and PyTorch’s scaled-dot-product attention dispatches to fused kernels of this kind. Multi-query attention [16] shrinks the KV cache read at every decode step. Our harness uses the stock attention path and does not modify kernels.

Precision, quantization, and compilation. FP16 mixed precision [13] and BF16 [11] were introduced mainly for training speed and numerical range; their relative latency at inference on a given GPU is an empirical question that we measure directly. Weight quantization such as LLM.int8() [9] and GPTQ [10] further reduces memory traffic; we leave it to future work. `torch.compile` [6] captures Python programs as graphs (TorchDynamo) and generates fused kernels (TorchInductor); we evaluate its default mode.

Benchmarking methodology. MLPerf Inference [15] standardizes inference measurement across systems, and the roofline model [18] relates achieved performance to compute and bandwidth limits. Our study is narrower: a controlled factor sweep on one consumer GPU, intended as a reproducible reference point rather than a benchmark suite.

3 Method

3.1 Software stack

Experiments use PyTorch [2] with CUDA 12.8 bindings (`torch 2.11.0+cu128`), Hugging Face Transformers [19] for model loading, and scaled-dot-product attention via PyTorch SDPA backends. The workload is greedy `model.generate()` with KV caching enabled. Synthetic token ids form prompts so logging of decoded text cannot contaminate timed regions.

3.2 Metrics and timing protocol

For each configuration we run a warmup phase followed by measured iterations. Each iteration records GPU-synchronized end-to-end latency of `generate()`, peak CUDA memory allocated, and tokens-per-second derived from generated token count and wall time. We report distributional summaries (mean, selected percentiles) rather than a single timing. Cold-start model load is outside the measured loop; `torch.compile` configurations still include steady measured iterations after warmup, but compilation cost can affect short windows and is treated as part of the system under test.

3.3 Experimental matrix

Model: HuggingFaceTB/SmolLM-135M [4], a 135M-parameter decoder-only language model. Factors: `dtype` $\in \{\text{fp16}, \text{bf16}\}$, batch size $\in \{1, 4\}$, prompt length $\in \{64, 256\}$, max new tokens $\in \{32, 128\}$, `compile` $\in \{\text{false}, \text{true}\}$. This yields 32 cells. Hardware: a single desktop workstation with an NVIDIA GeForce RTX 5070 (≈ 12 GB, NVIDIA 610-series driver) running Windows 11.

4 Results

All 32 cells completed with no OOM events. The lowest mean end-to-end latency was 586 ms (BF16, batch 4, prompt 256, gen 32, compile enabled; ≈ 218 tok/s). The highest was 2855 ms (FP16, batch 4, prompt 64, gen 128, eager; ≈ 180 tok/s).

4.1 Precision

Figure 1 compares FP16 and BF16 latency for every matched non-dtype factor. BF16 was faster in all 16 pairs. The median BF16/FP16 latency ratio was 0.895 (mean 0.890).

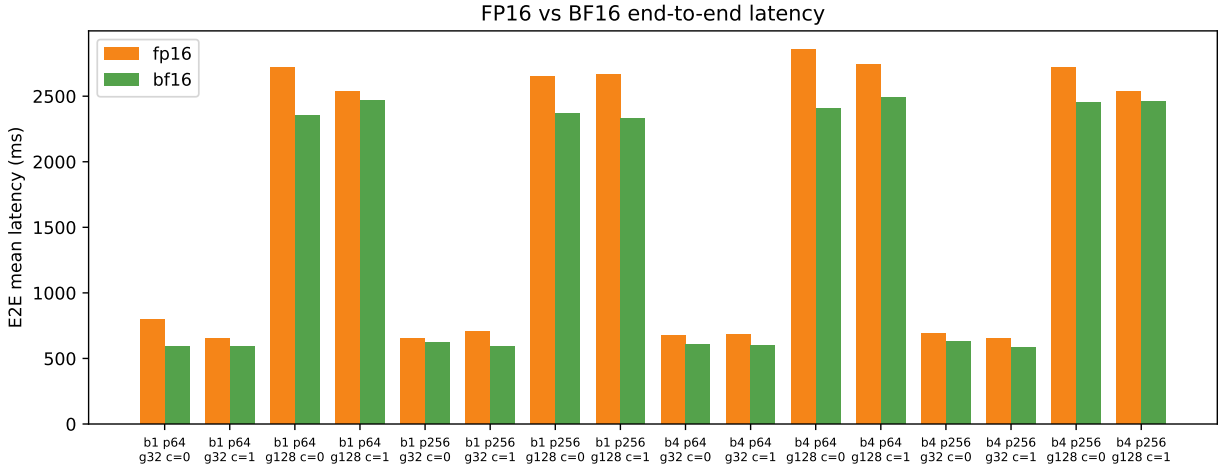


Figure 1: End-to-end mean latency for FP16 vs. BF16 across matched matrix cells.

4.2 Batching and throughput

Figure 2 shows tokens-per-second versus batch size for FP16 eager runs. Moving from batch 1 to batch 4 increased throughput from 41–49 tok/s to 180–189 tok/s. End-to-end latency did not rise with batch size in this range: going from batch 1 to batch 4 raised throughput without a latency penalty. With only two batch sizes, however, the matrix cannot show where that stops, so we ran a dedicated sweep (Section 4.3).

4.3 Batch-size sweep: the latency–throughput tradeoff

To locate where batching stops being free, we swept batch size over $\{1, 2, 4, \dots, 256\}$ with all other factors fixed (BF16, eager, prompt 256, 128 new tokens; 2 warmup and 5 measured iterations per cell). We ran the sweep three times. The first pass was inconsistent with the matrix (batch 1 took 3.4 s versus 2.37 s in the matrix, and batch 256 was faster than batch 128), which we attribute to host-side variability on the workstation during that pass; it is excluded from the aggregate but kept in the released analysis file. The second pass ran batch sizes in descending order and the third in ascending order; they agreed within 2% at every batch size, and Table 1 and Figure 3 report their mean. No cell ran out of memory; peak allocated memory was 4.0 GB at batch 256.

Two regimes are visible. From batch 1 to batch 32, end-to-end latency stays between 2.30 and 2.35 s (about 18 ms per decode step) while throughput rises from 56 to 1,754 tok/s. In this regime a decode step for this 135M-parameter model does not occupy the GPU, so per-step time is set by fixed costs that do not grow

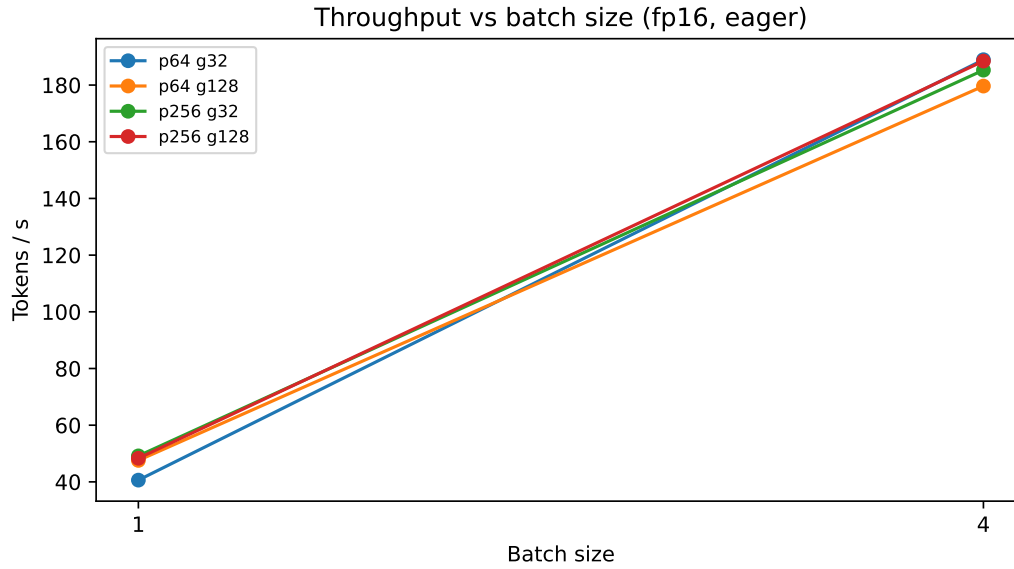


Figure 2: Throughput versus batch size (FP16, eager).

Batch	Latency (ms)	Throughput (tok/s)	Peak memory (MB)
1	2303	56	283
2	2314	111	299
4	2354	218	328
8	2335	438	386
16	2317	884	502
32	2336	1754	733
64	3687	2222	1197
128	6827	2401	2126
256	13286	2467	3980

Table 1: Batch-size sweep (BF16, eager, prompt 256, 128 new tokens), mean of two passes.

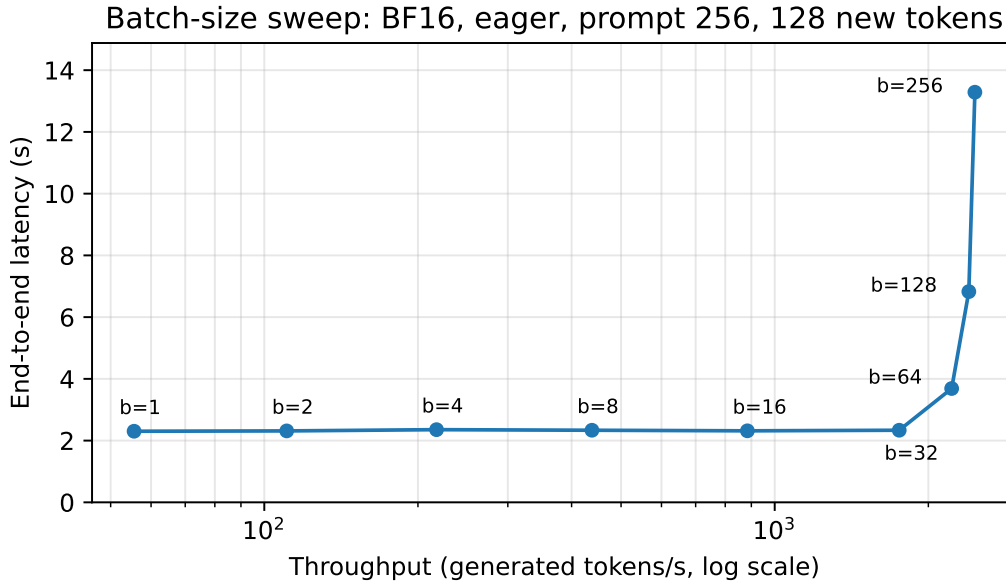


Figure 3: Latency versus throughput across the batch-size sweep. Each point is one batch size; the curve is flat up to batch 32 and then turns sharply upward.

with batch size, and additional sequences are nearly free. Beyond batch 32 the GPU becomes the bottleneck: latency grows roughly in proportion to batch size (3.7 s at 64, 6.8 s at 128, 13.3 s at 256) while throughput rises only from 2,222 tok/s at batch 64 to 2,467 tok/s at batch 256. For a latency target, the useful operating point on this hardware is therefore around batch 32; batch 64 buys about 27% more throughput at a 58% latency cost, and larger batches add little throughput. We did not profile the decode step, so the attribution of the flat regime to fixed per-step costs is an interpretation of the curve shape rather than a measured breakdown.

4.4 Generation length

Figure 4 plots latency against generated token count for BF16 eager batch 1. Increasing generation length from 32 to 128 raised end-to-end latency by roughly 4 $\times$, while tokens-per-second stayed between 52 and 54 tok/s. Under this harness, wall-clock time tracks decode length more strongly than the modest prompt-length change (64 vs. 256).

4.5 Compilation

Figure 5 reports eager/compile latency ratios. The median ratio was 1.015 (mean 1.027), ranging from 0.921 to 1.218. Thus `torch.compile` was approximately neutral on average for this model and measurement protocol, with occasional gains or regressions depending on the cell.

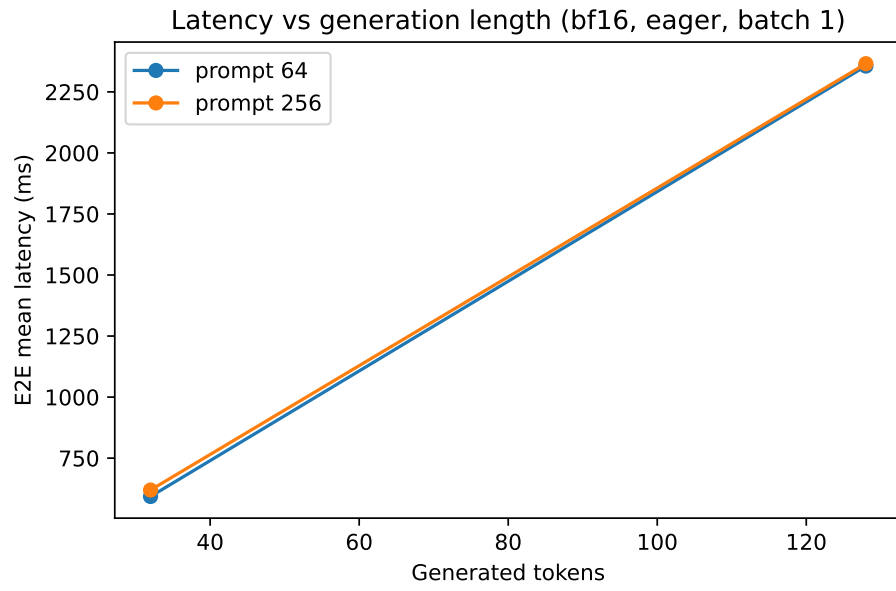


Figure 4: Latency versus generation length (BF16, eager, batch 1).

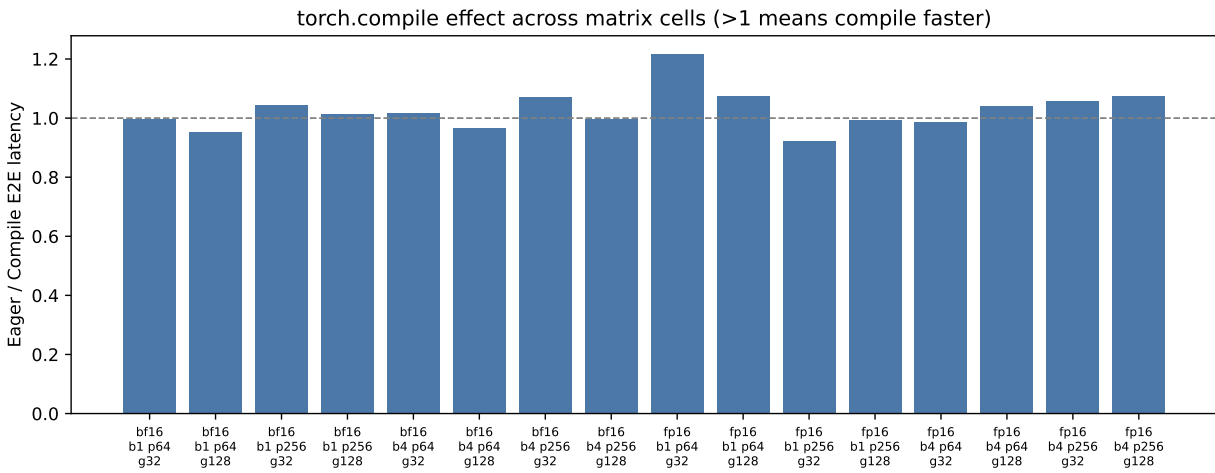


Figure 5: Eager/compile end-to-end latency ratios (> 1 means compile is faster).

5 Discussion and limitations

These measurements characterize one GPU, one small model, and an end-to-end `generate()` timer. They do not isolate time-to-first-token or inter-token latency, do not evaluate weight quantization or continuous batching under load, and do not claim optimality of any backend. The batch-size sweep uses a single configuration and reports its first, excluded pass openly; its two retained passes agreed closely, but results on a shared desktop workstation remain sensitive to host state. Within that scope, five conclusions are stable in the data: BF16 reduced latency vs. FP16; batching up to 32 raised throughput with no latency cost; beyond batch 32 latency grew roughly linearly while throughput saturated; generation length dominated wall time; and `torch.compile` did not provide a consistent speedup.

6 Conclusion

We measured a full 32-cell inference matrix and a batch-size sweep on an RTX 5070 using a reproducible harness. The sweep shows a clear latency-throughput knee near batch 32 for this model. The results emphasize workload- and precision-dependent behavior and caution against assuming that compilation alone improves short, small-model inference runs. A natural next step is to profile a decode step below and above the knee and to test CUDA Graphs, which remove per-step launch overhead and would therefore flatten the low-batch regime if our interpretation is correct.

Code and data availability

The harness, configurations, analysis scripts, and aggregated results are available at <https://github.com/benkarlsberg/GPU-Optimized-Transformer-Inference>.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.
- [2] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pages 8024–8035, 2019.
- [3] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [4] L. Ben Allal, A. Lozhkov, E. Bakouch, L. von Werra, and T. Wolf. SmolLM: Blazingly fast and remarkably powerful. Hugging Face blog, <https://huggingface.co/blog/smollm>, 2024.
- [5] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley, and Y. He. DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022.

- [6] J. Ansel et al. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.
- [7] T. Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.
- [8] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- [9] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- [10] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.
- [11] D. Kalamkar et al. A study of BFLOAT16 for deep learning training. arXiv:1905.12322, 2019.
- [12] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [13] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu. Mixed precision training. In *International Conference on Learning Representations (ICLR)*, 2018.
- [14] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 5, 2023.
- [15] V. J. Reddi et al. MLPerf inference benchmark. In *ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020.
- [16] N. Shazeer. Fast transformer decoding: One write-head is all you need. arXiv:1911.02150, 2019.
- [17] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- [18] S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [19] T. Wolf et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.
- [20] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.